Mars Operations

Welcome to Mars. Please Don't Die.

Benvenuto Ida Francesca 2045045
Casolino Ilaria 2079322
Festugato Luca 2064594
Giamberardini Matteo 2045678
Innocenzi Francesco 2087246

Mars Operations

Real-Time IoT Monitoring & Automation Control Center

- Simulates an **operations control center** for a Mars habitat environment.
- Connects to a **Mars IoT Simulator** producing live sensor readings and telemetry streams.
- Operators can monitor habitat conditions and **create custom automation rules**.
- Goals: **real-time monitoring**, **event-driven automation**, and **full operator control** — all in a containerized stack.

What the System Must Do — User Stories

- As an operator, I want the dashboard organized into **distinct visual sections** (sensors, telemetry, actuators) to navigate quickly.
- As an operator, I want the dashboard to **update in real-time**, without refreshing the page.
- As an operator, I want to see the **latest sensor values**, their units, and min/max peaks for the current session.
- As an operator, I want to **define, view, and delete automation rules** so the habitat reacts automatically.
- As an operator, I want to **manually toggle actuators** and view a complete **action history** log.
- As a developer, I want all services **containerized with Docker Compose** and communicating via a **standardized event schema**.

UI Layout — Three-Page Dashboard

The dashboard is divided into **three dedicated pages**, each serving a clear operational purpose:

- **Home page:** system health summary, active rules count, and real-time action history log.
- **Sensors & Telemetry page:** live charts, current values, units, min/max, and warning highlights.
- **Rules & Actuators page (/dashboard/rules):** list of all actuators with toggle controls, plus a form to create and delete automation rules with timestamps.
- Real-time updates flow into the UI via **WebSocket connections** — no page refresh needed.

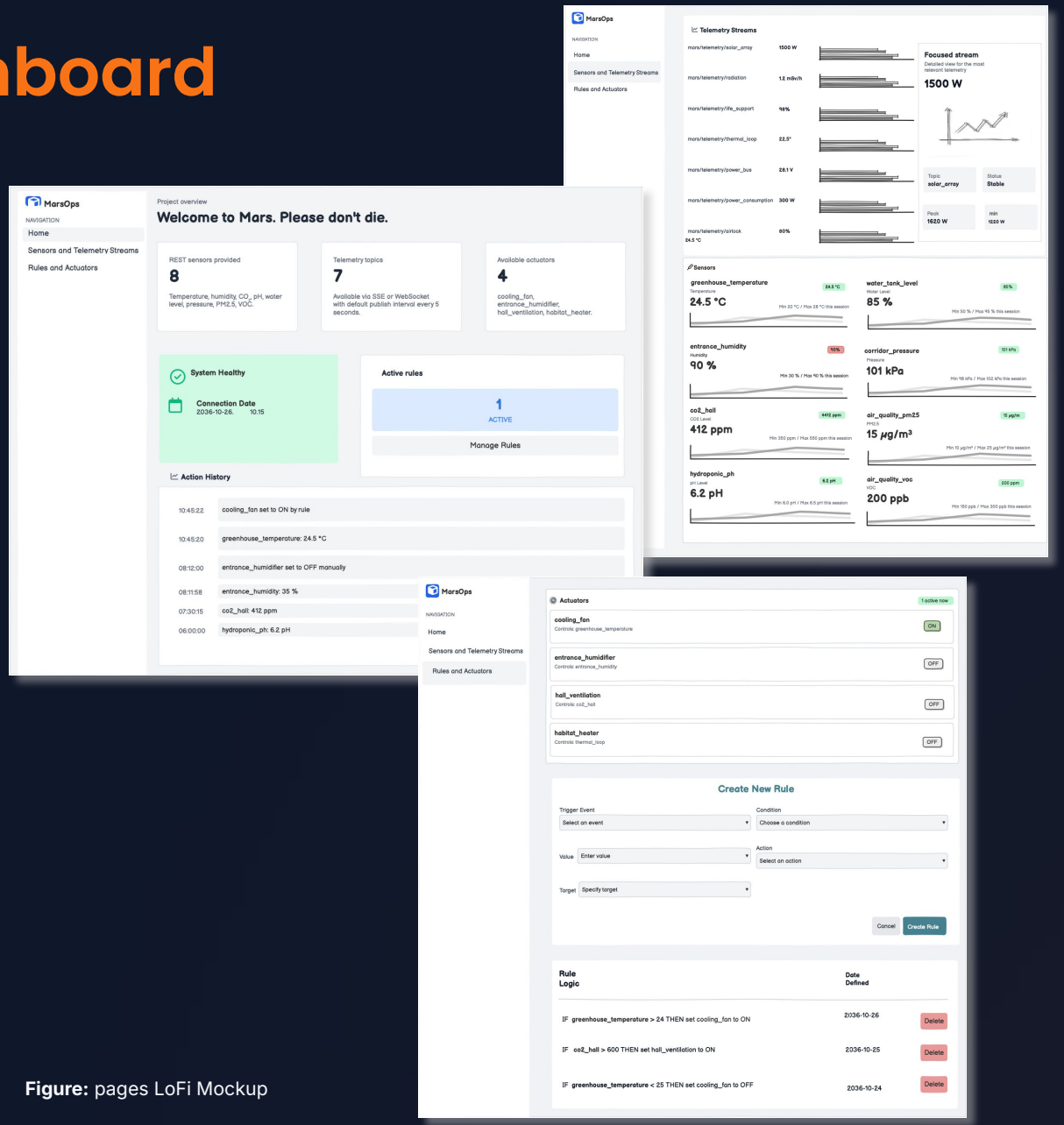


Figure: pages LoFi Mockup

Technology Stack — Motivation

Python + FastAPI: Selected for its native asynchronous support and Pydantic validation, enabling the rapid development of clean, high-performance REST and WebSocket APIs.

MariaDB: A relational database model that perfectly aligns with our structured business rules, utilizing SQL constraints to strictly enforce data integrity.

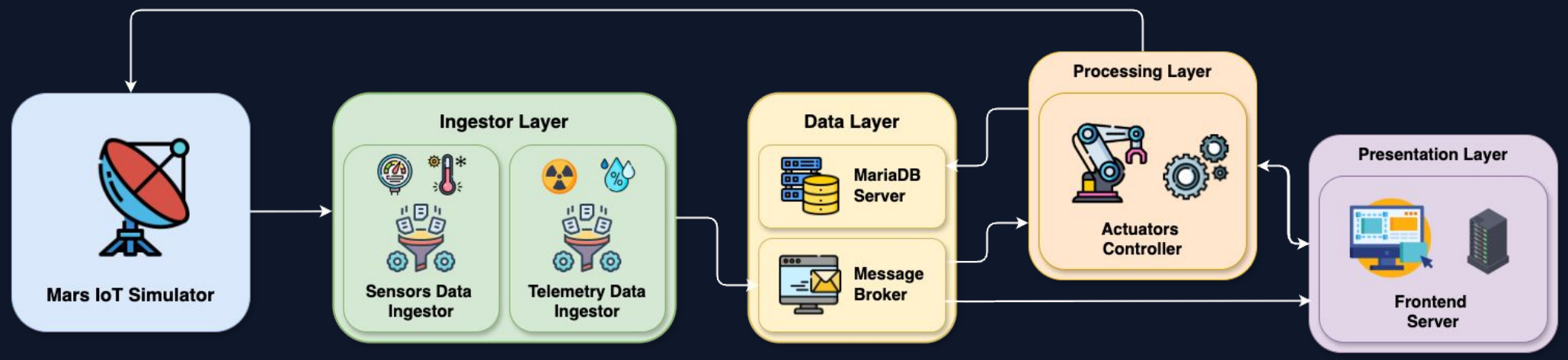
ActiveMQ: Leverages the STOMP protocol to effectively decouple message producers from consumers, ensuring reliable asynchronous communication.

Docker Compose: Deployment with a single-command startup, guaranteeing isolated and reproducible service environments across development and production.

HTML5 + CSS + JS: A lightweight, responsive dashboard. HTML provides the structural foundation, CSS handles responsive layouts and visual status alerts, and JavaScript drives real-time WebSocket data updates without page reloads.

System Architecture

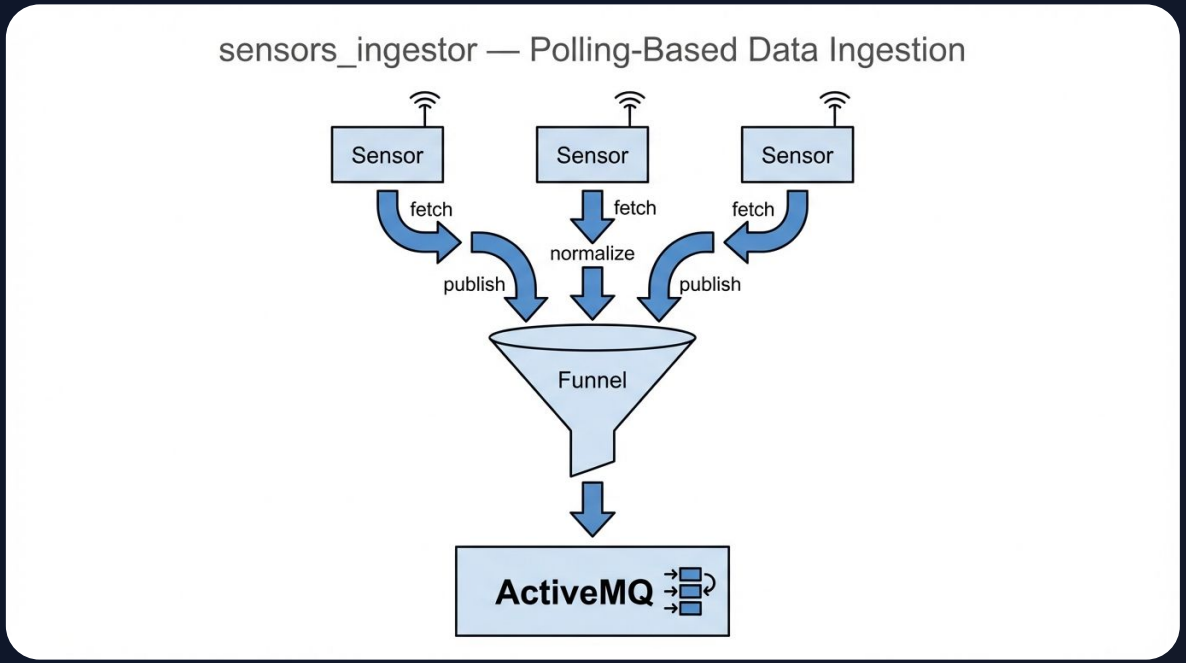
The system is built around a **central message broker** that decouples data ingestion from processing and presentation.



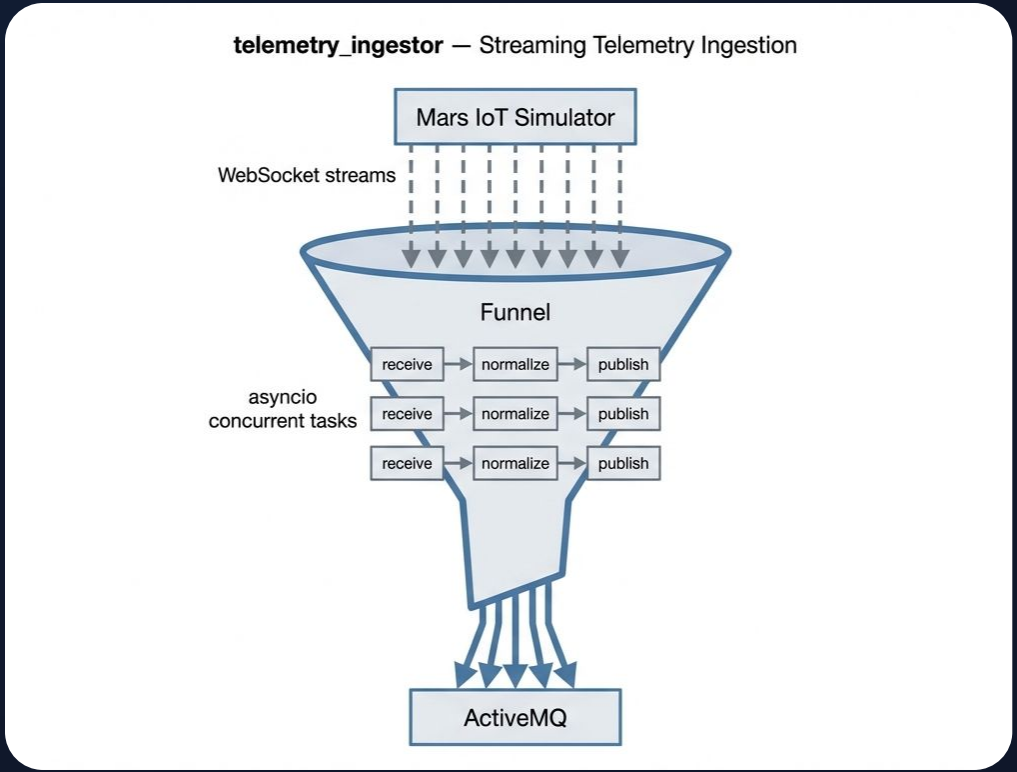
- **Mars IoT Simulator:** exposes sensor REST endpoints and telemetry WebSocket streams.
- **Ingestors:** collect raw data, normalize it, and publish standardized events to ActiveMQ.
- **ActiveMQ:** routes events to the Actuators Controller and Frontend Server.
- **Actuators Controller:** evaluates events against stored rules and sends commands back.
- **Frontend Server:** subscribes to broker events and pushes live data via WebSockets.
- **MariaDB:** stores automation rules persistently via Docker Compose.

sensors_ingestor — Polling-Based Ingestion

- Responsible for **polling the Mars IoT Simulator's REST endpoints** at regular intervals.
- Built as a **multi-threaded Python worker**: a dedicated daemon thread is spawned for each configured sensor. Each thread independently loops: `fetch → normalize → publish`
- Raw heterogeneous JSON payloads are **normalized into the Unified Event Schema** before being published.
- Events are pushed to **ActiveMQ via the STOMP protocol**.



telemetry_ingestor — Async Stream Ingestion



- Connects to the Mars IoT Simulator via **persistent WebSocket connections** to receive continuous telemetry streams.
- Built with an **asyncio-based cooperative architecture**: a single thread manages multiple concurrent tasks, one per telemetry topic.
- Each incoming message is **normalized into the Unified Event Schema** and published to ActiveMQ.
- **7 telemetry streams**: solar array, radiation, life support, thermal loop, power bus, power consumption, and airlock.

Standardized JSON Event Schema

```json
{
  "type": "object",
  "required": ["device_id", "time", "metrics"],
  "properties": {
    "device_id": {"type": "string"},
    "time": {"type": "string", "format": "date-time"},
    "status": {"type": "string", "enum": ["ok", "warning"]},
    "metrics": {
        "type": "array",
        "items": {
        "type": "object",
        "required": ["metric_name", "value"],
        "properties": {
          "metric_name": {"type": "string"},
          "value": {"type": "number"},
          "unit": {"type": "string"}
        }
      }
    },
    "metadata": {
        "type": "object",
        "properties": {}
    }
  }
}
```

One Schema, Universal Understanding

A single shared schema ensures all components speak the same language.

- **`device_id`** + **`time`** are mandatory on every event.
- **`status : warning`** triggers a **visual highlight** on the dashboard.
- **`metrics`** is an array — one event can carry multiple measurements simultaneously.
- **`metadata`** is an object storing all the additional fields that are not included in previous ones.

ActiveMQ

Central Event Bus

ActiveMQ fully decouples data producers from consumers. Both ingestors publish events to dedicated **topics** (one per sensor or telemetry stream).

Routing & Consumers

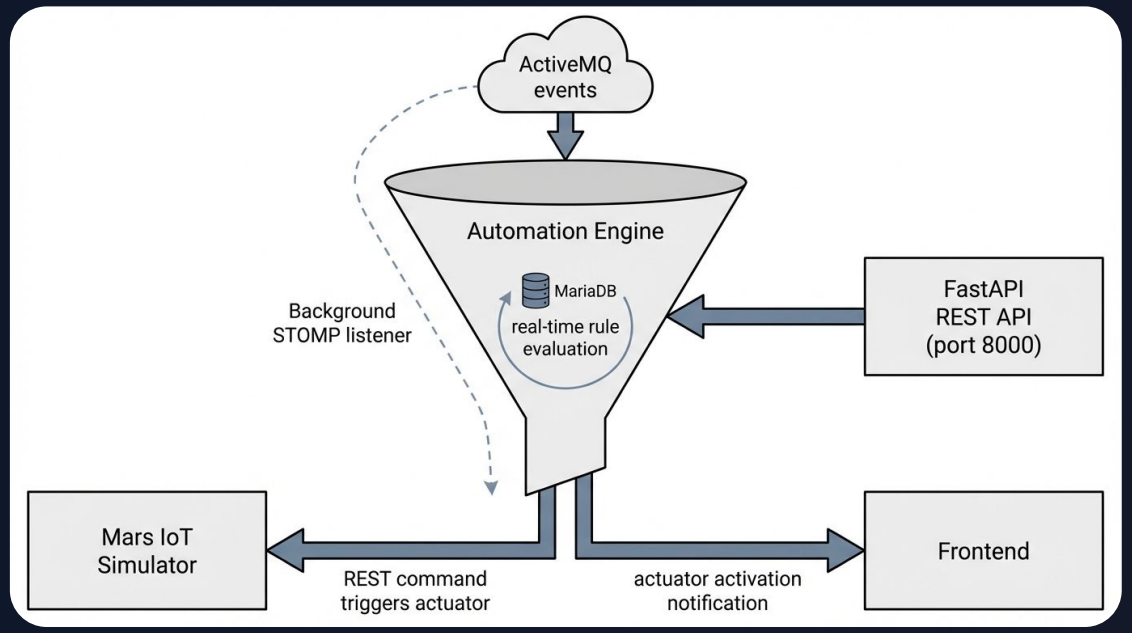
The **Actuators Controller** subscribes to receive sensor data. The **Frontend Server** subscribes to receive sensor and telemetry data and forward them to connected browsers.

STOMP & Scalability

Uses **STOMP protocol** (port 61613). Adding a new consumer requires **zero changes** to the producers.

actuators_controller — Automation Engine & API Gateway

- Dual role: **FastAPI REST server** (port 8000) and **background STOMP event listener**.
- For every incoming event, **rules from MariaDB** are evaluated in **real time**.
- On rule match, a REST command is sent to the **Mars IoT Simulator** to trigger the actuator.
- The frontend is **notified** of the activation via a POST to `/activate_actuator`.
- Endpoints: `POST /create_rule`, `POST /delete_rule/{id}`, `GET /rules`, `POST /toggle_actuator/{name}`.



mariadb — Persistent Rule Storage

MariaDB is the **only stateful component** in the system, storing automation rules that must survive container restarts.

Table: automation_rules

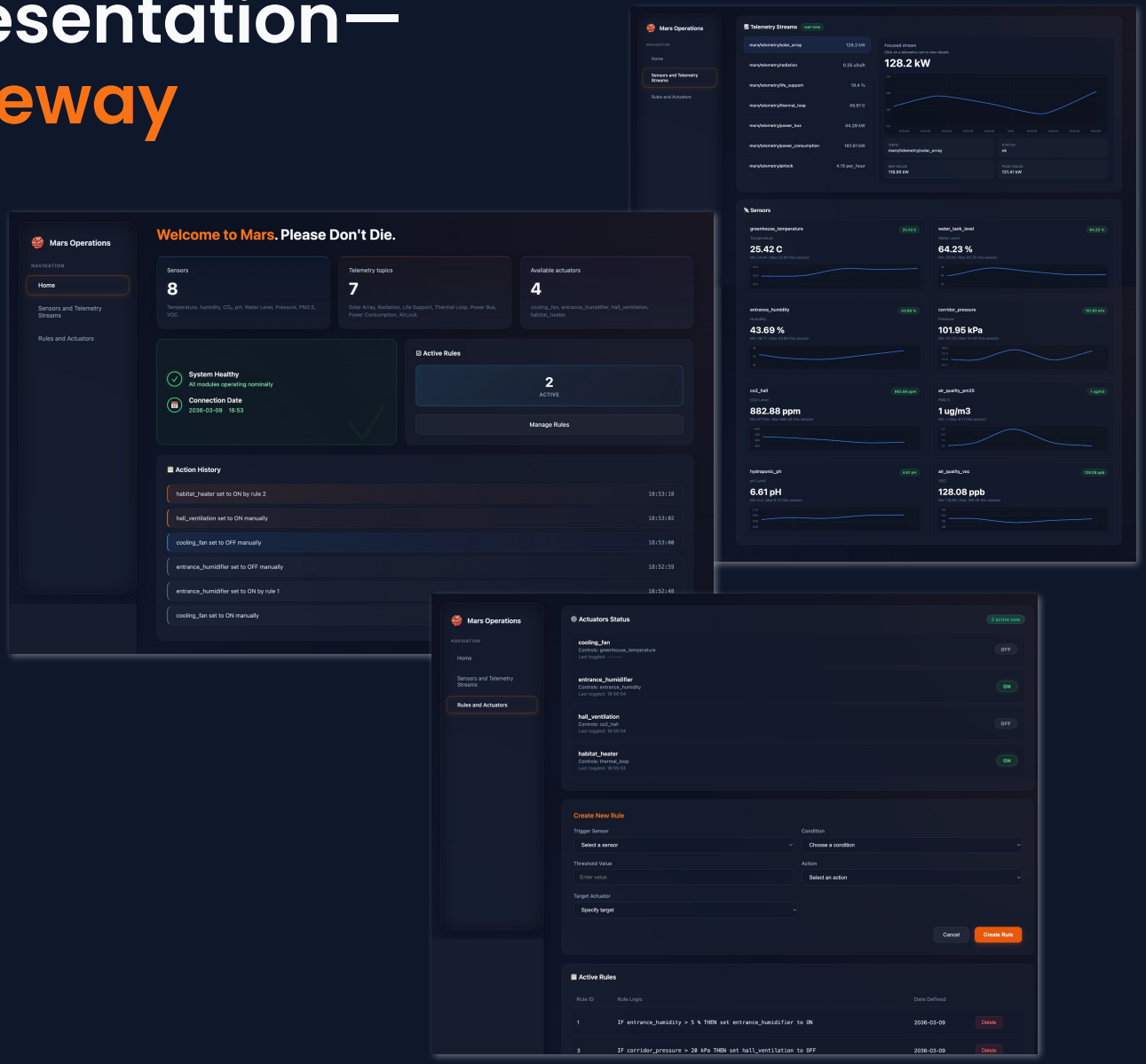
Column	Type
sensor_name	VARCHAR(100)
operator	ENUM(`<`,`<=`,`=`,`>`,`>=`)
threshold_value	FLOAT
actuator_name	ENUM(4 devices)
action	ENUM(`ON`,`OFF`)
created_at	TIMESTAMP

- A **UNIQUE constraint** on **(sensor_name, actuator_name, action)** prevents duplicate rules.
- Data persisted via a **Docker named volume**.

Frontend server and User Presentation—
Dashboard & Real-Time Gateway

- Built with **FastAPI + Jinja2**, serving three HTML pages to the browser (port 8005).
- Acts as a **proxy**: operator actions from the client browser are forwarded to the `**actuators_controller**`.
- Real-Time data flows via WebSocket endpoints. JavaScript connect to it, then updates charts, value, and warning live.

The dashboard allows users to interact directly with the system, modifying rules and observing parameter changes in real time.



Team Workflow & Task Distribution

Microservices First

Developed following a microservices approach: each container was designed, implemented, and tested independently before integration.

Docker & Git Sync

The **docker-compose.yaml** served as the integration contract. Git was implemented for robust version control, enabling seamless parallel development across the team while ensuring a structured and controlled evolution of the codebase.

Documentation

Maintained throughout: `input.md` for requirements, `Student_doc.md` for container specs, and the **LoFi Mockups Booklet** for UI design.

Mars Operations

Welcome to Mars. Please Don't Die.